BLE-Based Indoor Environmental Monitoring System using Nordic nRF52840 Boards

Group Project
Safety Critical Systems
Summer Semester 2026

May 18, 2026

Contents

1 Introduction

Indoor air quality directly affects human health, cognitive performance, and occupational safety. Because traditional monitoring approaches often rely on periodic manual inspection or single-parameter threshold alarms, they are typically spatially limited and analytically shallow. Reliable, intelligent indoor monitoring systems have become increasingly important for modern indoor environments.

To address these challenges, this project proposes a Bluetooth Low Energy (BLE)-Based Indoor Environmental Monitoring System. Built on a distributed IoT architecture, the system utilizes Nordic nRF52840 development boards and an advanced environmental sensor suite (SCD41, SPS30, CCS811) to continuously monitor CO_2, particulate matter, VOCs, temperature, and humidity.

Sensor nodes periodically gather multi-dimensional environmental measurements and transmit them via BLE advertising to a gateway. The data is then ingested by a containerized microservices backend, processed through an unsupervised machine learning anomaly detection layer, and visualized on a real-time user dashboard.

1.1 Project Scope

This project focuses on the end-to-end design and implementation of a distributed intelligent indoor environmental monitoring network. The scope encompasses the following core development areas:

- **Embedded Systems:** Firmware development for Nordic nRF52840-based sensor nodes responsible for multi-dimensional environmental data acquisition and BLE Advertising.
- **Gateway Integration:** Implementation of a Python-based BLE-to-HTTP gateway to bridge wireless sensor telemetry into the software backend.
- **Software and Data Processing:** Development of a containerized Spring Boot backend for data ingestion and a MongoDB database for persistent storage.
- **Machine Learning Analytics:** Integration of an unsupervised anomaly detection service (Isolation Forest) to identify complex behavioral anomalies in the telemetry stream.
- **Presentation:** Development of a real-time React-based dashboard for data visualization and unified alert management.

The architecture is designed to support multiple BLE sensor nodes operating across different indoor locations. Due to current hardware availability constraints, the initial implementation and validation phase will use one fully instrumented sensor node while preserving multi-node scalability at the communication, backend, and dashboard layers. The project emphasizes reliable communication, robust microservice design, and the application of software engineering best practices throughout the development life cycle.

1.2 Major Software Functions

The system software architecture is distributed across five primary components: the sensor nodes, the BLE gateway, the backend REST API, the machine learning analytics service, and the monitoring dashboard. Each component operates as an isolated containerized service (except the physical sensor nodes) and is responsible for a specific set of software functions.

- **Sensor Node Firmware (Nordic nRF52840 DK):** Responsible for periodic environmental data acquisition using a five-sensor suite (SCD41, SPS30, CCS811, MCP9808, KY-018). The firmware handles BLE packet generation, sensor data formatting, and wireless transmission via BLE Advertising.

- **BLE Gateway Subsystem (Python / Bleak):** Acts as the central communication bridge. The gateway continuously scans for BLE telemetry packets from distributed sensor nodes and forwards the collected environmental data to the backend server via HTTP REST requests.
- **Backend Server (Spring Boot):** Handles data ingestion, persistent storage in MongoDB, and evaluates deterministic safety thresholds. The backend system also manages node status tracking and provides the primary API for the dashboard.
- **Machine Learning Analytics Service (FastAPI):** Operates asynchronously on the live telemetry stream. It applies an Isolation Forest anomaly detection model to identify statistically anomalous multi-variable patterns and dispatches alerts to the backend.
- **Frontend Dashboard (React):** Provides a graphical monitoring interface for administrators. The dashboard visualizes real-time telemetry data, historical environmental trends, sensor node status, and a unified feed combining both threshold and ML-based alerts.

1.3 Performance and Behavioral Requirements

The system is expected to maintain reliable operation despite the limitations of low-power wireless hardware and BLE-based communication. Since the proposed architecture relies on distributed sensor nodes and wireless telemetry, several performance and behavioral considerations must be addressed during implementation and testing.

- **Wireless Communication Reliability:** BLE communication should remain stable under conditions such as packet loss, communication interruptions, and signal interference commonly present in indoor wireless environments.
- **Resource Efficiency:** Embedded sensor nodes must operate efficiently with limited computational resources and low power consumption while maintaining stable periodic sampling and data transmission intervals.
- **Low-Latency Processing:** The backend infrastructure should process incoming telemetry data with minimal delay to support near real-time environmental monitoring, anomaly detection, and system status tracking.
- **Graceful Degradation and Fault Tolerance:** The monitoring system should continue operating even if individual sensor nodes become unavailable or disconnected. Basic fault-tolerance mechanisms and fault-detection logic should ensure that failures in a single node do not compromise the overall functionality of the distributed monitoring network.

1.4 Safety, Security, and Reliability Targets

Because inaccurate or unavailable environmental data may lead to incorrect assumptions regarding indoor conditions, the system emphasizes reliability, fault detection, and secure communication throughout the monitoring process.

- **Safety and Failure Handling:** A primary safety objective is preventing "silent failures," where outdated sensor readings continue to be displayed after data collection or communication has stopped. If a sensor node becomes unavailable or disconnected, the system should explicitly indicate the loss of connectivity rather than presenting stale environmental data.
- **Reliability and Fault Recovery:**

  - **Firmware Watchdog Mechanisms:** Embedded firmware running on the sensor nodes utilizes watchdog timers to automatically recover from potential software deadlocks or unexpected execution failures.
  - **Timeout Monitoring and Alerts:** The backend processing layer continuously monitors sensor node activity and triggers alerts if telemetry data is not received within predefined timeout intervals.

- **Security and Access Control:**
  - **Node Authorization:** The gateway accepts communication only from authorized BLE sensor nodes using predefined device identifiers and MAC address filtering mechanisms.
  - **Backend Authentication:** Access to the backend server, database, and monitoring dashboard is protected through authentication mechanisms to reduce the risk of unauthorized data access or modification.

1.5 Management and Technical Constraints

The design, implementation, and deployment of the proposed monitoring system are subject to several management and technical constraints that influence the final system architecture and development process.

- **Hardware Constraints:** Due to current hardware availability limitations, the system will initially be validated using a single fully instrumented sensor node equipped with a five-sensor suite (SCD41, SPS30, CCS811, MCP9808, KY-018) rather than multiple partially equipped nodes. The gateway functionality is provided by a Bluetooth-capable host machine running Python.
- **Technical Constraints:** The system architecture must account for the inherent limitations of BLE communication and embedded hardware resources. These limitations include restricted wireless communication range, potential signal interference, limited processing capability of embedded nodes, and the accuracy constraints associated with low-cost environmental sensors.
- **Development Timeline and Project Management:** The project is constrained by a 14-week semester development schedule. Since the implementation spans multiple technical domains, including embedded systems, wireless networking, backend infrastructure, and frontend visualization, successful project delivery requires effective task distribution, sprint-based planning, and coordinated team collaboration.

1.6 Process Model

The project follows an Agile software development methodology using the Scrum framework. Development activities are organized into iterative two-week sprint cycles, enabling incremental implementation, integration, testing, and validation of system components throughout the semester.

The Scrum process supports adaptive planning, continuous progress tracking, and collaborative development among the six project members. Team responsibilities are distributed across embedded systems development, backend infrastructure, frontend visualization, system integration, testing, and project coordination activities.

Software Configuration Management (SCM) practices are implemented using Git-based version control with dedicated feature branches and structured integration workflows. Jira is used for sprint planning, backlog management, task assignment, and project progress tracking.

Sprint reviews and retrospectives are conducted regularly to evaluate technical progress, identify development challenges, and continuously improve the development process during the project lifecycle.

2 Project Estimates

This section provides effort, workload, and resource estimations for the BLE-based indoor environmental monitoring system project.

2.1 Estimation Assumptions and Baselines

The project estimation is based on assumptions derived from published IoT and wireless sensor network (WSN) literature, standard software engineering practices, and the technical constraints of embedded system development.

- **Historical Baselines:** Previous studies on embedded IoT systems indicate that system integration, debugging, communication reliability, and hardware validation typically consume a significant portion of the total development effort.
- **Technical Complexity Factors:** Hardware-software integration activities such as BLE communication setup, gateway synchronization, sensor calibration, backend integration, and fault-handling mechanisms are considered high-effort development tasks. Additional effort is therefore allocated for integration testing, communication validation, and system stabilization.
- **Resource Capacity:** The project is constrained by a fourteen-week semester timeline. Assuming a six-member development team contributing approximately ten hours per week, the total estimated project effort is approximately 840 person-hours.

2.2 Estimation Techniques Applied and Results

For this project, a hybrid estimation strategy combining Bottom-Up Estimation and Expert Judgment was adopted.

2.2.1 Bottom-Up Estimation

The system was decomposed into manageable development activities including sensor integration, BLE networking, Python gateway development, Spring Boot backend development, dashboard implementation, Machine Learning analytics integration, testing, and documentation. Effort estimates were assigned to each subsystem and later combined to form the overall project estimate.

2.2.2 Expert Judgment

Estimates were refined using prior development experience and technical complexity assessments regarding the complexity of Nordic nRF52840 BLE development, backend integration, debugging effort, and software testing activities. Greater effort allocation was assigned to wireless communication and system integration due to their higher technical risk.

2.2.3 Estimated Workload Distribution

The total estimated workload for the semester project is approximately 840 person-hours.

Table 1: Estimated Workload Distribution

Project Activity	**Percentage**	**Estimated Hours**
Hardware Interfacing and Sensor Integration	15%	126
BLE Communication and Python Gateway	15%	126
Backend Platform and Data Pipeline	15%	126
Frontend Dashboard Development	15%	126
ML Analytics and Anomaly Detection	15%	126
Testing, Validation and Debugging	15%	126
Documentation and Project Management	10%	84
Total	**100%**	**840**

2.3 Reconciled Estimate

After comparing the results from Bottom-Up Estimation and Expert Judgment, the reconciled estimate for the project is considered achievable within a fourteen-week semester schedule.

The project benefits from having all required hardware components already available, reducing procurement delays and allowing the team to focus on implementation, integration, and testing activities during the early project phases.

Additional schedule margin has been reserved for BLE debugging, hardware calibration, and system-level verification because these tasks typically require iterative testing and troubleshooting.

2.4 Project Resources

2.4.1 Hardware Resources

The project utilizes the following hardware resources:

- 1x Nordic nRF52840 Development Kit
- 1x Sensirion SCD41 CO_2, Temperature, and Humidity Sensor
- 1x Sensirion SPS30 Particulate Matter Sensor
- 1x Adafruit CCS811 TVOC and eCO_2 Sensor
- 1x Microchip MCP9808 Temperature Sensor
- 1x KY-018 Light Intensity Sensor
- Bluetooth-capable host workstation

2.4.2 Software Resources

The following software technologies and tools are planned for development and testing:

- Nordic nRF Connect SDK
- Zephyr RTOS
- Visual Studio Code
- Git and GitHub for Software Configuration Management
- Spring Boot backend framework

- MongoDB or other NoSQL database
- Dashboard visualization framework
- Overleaf for LaTeX documentation

3 Safety and Security Plan

3.1 Safety Objectives

The primary safety objective of the monitoring system is to ensure reliable and continuous environmental monitoring without silent system failures. The system must immediately indicate when valid sensor measurements are unavailable or communication failures occur.

The monitoring architecture is designed to provide fault awareness rather than assuming uninterrupted operation. This is particularly important in distributed wireless sensor systems where hardware failures, communication interruptions, or sensor malfunctions may occur unexpectedly.

Importantly, the Machine Learning subsystem operates as a supplementary analytics layer and is not a safety-critical component. Deterministic threshold alerts are evaluated independently in the backend and remain fully operational if the ML service is unavailable or undergoing model recalibration. This separation ensures that core occupant safety guarantees are never contingent on ML subsystem availability.

3.2 Fault Detection Mechanisms

The backend processing system implements several fault detection strategies:

- Timeout monitoring for disconnected sensor nodes
- Detection of frozen or repeated sensor measurements
- Communication failure monitoring between BLE nodes and gateway
- Validation of received sensor payload structure

If a sensor node fails to transmit data within a predefined time interval, the backend marks the node as unavailable and generates a warning within the dashboard interface.

3.3 Reliability Considerations

System reliability depends on stable operation across the distributed architecture layers:

- Sensor node operation
- BLE wireless communication
- Gateway forwarding functionality
- Backend server availability
- Dashboard visualization services

To improve operational stability, watchdog timers may be implemented on the nRF52840 boards to automatically recover from firmware execution failures.

3.4 Communication Reliability

The project utilizes a BLE star-topology network where sensor nodes communicate directly with the gateway node. Wireless communication reliability is influenced by signal interference, physical obstacles, and transmission range limitations.

To reduce communication interruptions, the system incorporates:

- Automated BLE reconnection logic
- Periodic node heartbeat transmission
- Continuous gateway monitoring
- Error handling for malformed packets

3.5 Sensor Reliability

Environmental sensors may exhibit measurement inaccuracies and calibration drift over time. Prior to deployment, baseline calibration procedures and environmental stabilization will be performed for the 5-sensor suite, particularly the SCD41, SPS30, and KY-018 sensors.

The backend processing layer additionally performs basic filtering to reject physically impossible measurement values and reduce transient noise spikes.

3.6 Security Considerations

The security strategy focuses on maintaining communication integrity and controlled system access.

- BLE communication is restricted to authorized sensor nodes
- Gateway acceptance is limited to known device addresses
- Backend dashboard access requires authentication
- Logged environmental data is protected against unauthorized modification

3.7 Failure Handling Strategy

The architecture is designed for graceful degradation. If one sensor node becomes unavailable, the remaining nodes continue operation independently.

The backend server isolates failed nodes while maintaining monitoring availability for active nodes. Dashboard notifications inform users about degraded operational states and communication failures.

4 Risk Management and Analysis

4.1 Overview

Risk management is an essential component of safety-critical systems engineering. The purpose of this analysis is to identify technical, hardware, software, communication, and project-management risks that could affect the successful development or operation of the monitoring system.

4.2 Project Risk Assessment

Table 2: Project Risk Assessment

Risk Category	Risk Description	Possible Impact	Mitigation Strategy	Probability	Severity
Technical Risks	Complexity of BLE integration and wireless debugging	Development delays and unstable communication	Early BLE prototyping and use of Nordic SDK examples	Medium	High
Hardware Risks	Sensor inaccuracies and calibration drift	Incorrect environmental measurements	Baseline sensor calibration and software filtering	High	Medium
Software Risks	Integration bugs between distributed subsystems	Backend instability or incorrect dashboard behavior	Git-based configuration management and peer reviews	Medium	High
Communication Risks	BLE packet loss and connection interruptions	Missing telemetry and delayed alerts	Heartbeat monitoring and automatic reconnection logic	Medium	High
Safety Risks	Undetected sensor malfunction or frozen data	Silent monitoring failure and incorrect system state	Timeout detection and watchdog monitoring	Low	Critical
Project Management Risks	Semester timeline limitations	Incomplete implementation or reduced testing coverage	SCRUM sprint planning and milestone tracking	Medium	Medium

4.3 Technical Risk Discussion

The most significant technical challenge is the implementation and stabilization of the BLE communication infrastructure. Wireless communication systems often require extensive debugging due to synchronization problems, signal interference, and device compatibility issues.

Additionally, the integration of unsupervised Machine Learning (Isolation Forest) introduces new technical risks, such as model false positives and environmental concept drift (where changing seasons or sensor relocations trigger false anomalies).

To reduce integration risk, early development sprints focus specifically on hardware and BLE communication validation before expanding the architecture to backend processing, dashboard visualization, and ML analytics integration.

4.4 Safety Risk Discussion

The most critical safety-related risk is silent failure, where the system appears operational while valid environmental measurements are no longer being received.

The architecture addresses this risk using timeout detection mechanisms, heartbeat monitoring, and backend-side fault analysis capable of identifying disconnected or malfunctioning nodes.

4.5 Project Management Strategy

The SCRUM process model enables iterative development and incremental validation throughout the semester timeline. Weekly progress reviews, sprint planning sessions, and milestone tracking are used to reduce scheduling risk and improve implementation visibility across the development team.

5 Architecture Discussion

5.1 System Overview

The proposed system follows a distributed wireless environmental monitoring architecture utilizing Bluetooth Low Energy (BLE) communication and Nordic nRF52840 hardware.

The architecture is divided into five primary layers:

1. Sensor Layer
2. Gateway Layer
3. Backend Processing Layer
4. Analytics Layer
5. Visualization Layer

5.2 Sensor Layer

The sensing layer consists of Nordic nRF52840 Development Kits operating as distributed wireless sensor nodes.

Each node is responsible for acquiring multi-dimensional environmental data using a five-sensor suite:

- Acquiring CO_2, temperature, and humidity using the Sensirion SCD41
- Acquiring particulate matter (PM1.0/2.5/4.0/10) using the Sensirion SPS30
- Acquiring TVOC and eCO_2 using the Adafruit CCS811
- Acquiring high-precision thermal baselines using the MCP9808
- Acquiring ambient light intensity using the KY-018 sensor
- Packaging telemetry data into structured BLE payloads
- Periodically transmitting measurements via BLE Advertising

The sensor nodes operate independently and periodically communicate with the gateway node using BLE advertising or BLE peripheral communication.

5.3 Communication Layer

The communication layer utilizes a BLE star-topology architecture.

In this topology:

- Sensor nodes operate as BLE Advertising broadcasters
- A host machine running the Python Bleak client acts as the BLE central gateway
- All sensor data is captured and forwarded to the backend through the software gateway
- Inter-node communication is not required in the selected star-topology architecture.

The BLE star topology simplifies communication management and reduces routing complexity compared to mesh-based architectures.

5.4 Gateway Layer

The gateway layer acts as a bridge between the wireless BLE domain and the HTTP-based backend. It is implemented as a Python 3.12 microservice utilizing the `Bleak` library.

The gateway is responsible for:

- Scanning for BLE sensor advertisements
- Decoding sensor payloads
- Validating received telemetry formatting
- Forwarding measurements to the backend server via HTTP REST POST requests
- Monitoring node connectivity status

This software-based gateway abstracts the hardware layer, ensuring that switching from simulated to physical BLE hardware requires no downstream changes.

5.5 Backend Processing Layer

The backend server is a Java 21 Spring Boot 3.3 microservice that performs centralized data ingestion and orchestration. It utilizes MongoDB 7 for persistent document storage.

Primary responsibilities include:

- REST API telemetry ingestion
- Evaluating deterministic safety thresholds
- Database logging to MongoDB
- Timeout monitoring and fault detection
- Managing node status and alert generation
- Exposing endpoints for the dashboard and ML service

The backend continuously evaluates communication status to identify disconnected nodes.

5.6 Analytics Layer

The analytics layer introduces intelligent, multi-variable anomaly detection to the system. It is implemented as a FastAPI Python microservice utilizing `scikit-learn`.

Primary responsibilities include:

- Asynchronously polling the backend telemetry stream
- Training a per-node Isolation Forest unsupervised machine learning model
- Evaluating the joint distribution of all 5 sensor readings simultaneously
- Identifying subtle, cross-parameter behavioral anomalies
- Dispatching `ML_ANOMALY` alerts to the backend

This layer creates a hybrid alerting architecture, complementing deterministic safety rules with proactive environmental intelligence.

5.7 Dashboard Visualization Layer

The presentation layer is a React 19 single-page application served via Nginx. The dashboard interface displays:

- Real-time environmental telemetry charts
- Historical environmental trends
- Active sensor node connectivity status
- A unified alert feed combining deterministic and ML-triggered warnings
- Communication failure notifications

The visualization layer improves monitoring transparency and assists users in identifying abnormal environmental conditions.

5.8 System Data Flow

The complete system data flow is summarized as follows:

1. Sensor nodes acquire environmental measurements
2. nRF52840 nodes package sensor data
3. BLE payloads are transmitted to the gateway
4. Gateway forwards telemetry to the backend server
5. Backend stores and processes the data
6. Dashboard visualizes measurements and alerts

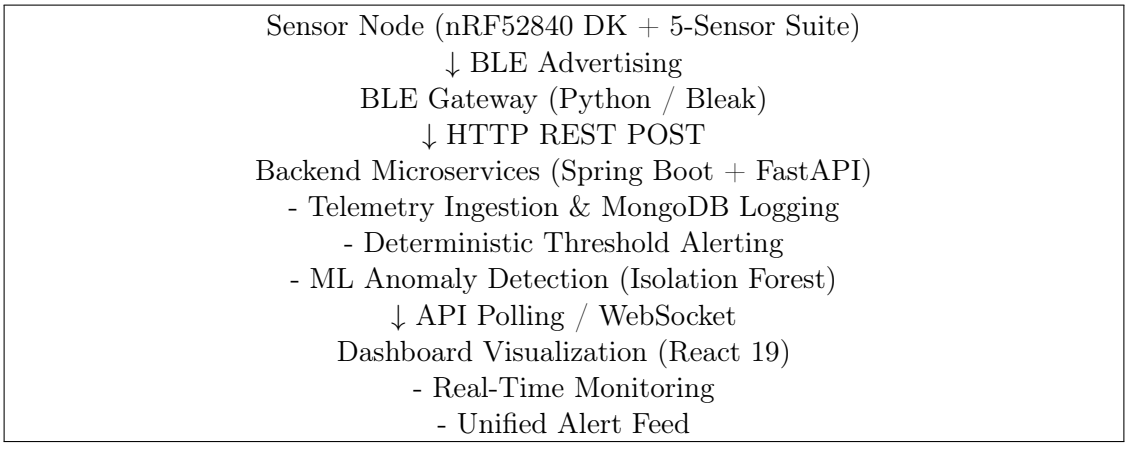


Figure 1: BLE-Based Smart Indoor Environmental Monitoring System Architecture

6 Project Schedule

6.1 Project Task Set

The project task set defines the major engineering activities required for the development of the BLE-based indoor environmental monitoring system.

- Hardware setup and multi-sensor interfacing
- BLE communication and payload formatting
- Python BLE gateway and HTTP REST forwarding
- Backend microservice implementation using Spring Boot and MongoDB
- Dashboard visualization and unified alert feed using React
- Machine Learning anomaly detection using Isolation Forest
- Testing and validation
- Documentation and final presentation

6.2 Functional Decomposition

The project is divided into several functional subsystems:

1. Sensor acquisition subsystem
2. BLE communication subsystem
3. Python Gateway forwarding subsystem
4. Backend processing and database subsystem
5. Machine Learning analytics subsystem
6. Dashboard visualization subsystem

Each subsystem can be developed and tested independently before full system integration.

6.3 Task Network

The development activities follow a dependency-based sequence.

- Sensor interfacing must be completed before BLE transmission testing.
- BLE communication must function before gateway integration.
- Gateway communication must be operational before backend ingestion testing.
- Database integration is required before dashboard visualization.
- System validation depends on the successful integration of all previous components.

This dependency structure minimizes integration risks and supports iterative SCRUM development.

6.4 Timeline Chart

The project follows a fourteen-week semester schedule divided into SCRUM sprints.

Table 3: Semester Development Schedule

Weeks	**Sprint / Phase**	**Main Activities**
1–2	Project Initialization	Hardware familiarization, literature review, architecture design, repository setup
3–4	Sprint 1 – Hardware and BLE Communication	Sensor integration, firmware development, BLE Advertising, gateway telemetry validation
5–6	Sprint 2 – Backend Platform and Data Pipeline	Docker orchestration, MongoDB, REST API, telemetry ingestion, threshold alerting
7–8	Sprint 3 – Dashboard and Integration	React dashboard, telemetry visualization, node status, alert feed, distributed service integration
9–10	Sprint 4 – ML Analytics and Finalization	Anomaly detection integration, system testing, documentation, final hardware integration
11–12	Sprint 5 – Testing and Validation	End-to-end integration, full pipeline testing, ML anomaly validation
13–14	Sprint 6 – Finalization	Bug fixing, report completion, presentation preparation

7 Staff Organization

7.1 Team Structure

The project team consists of six members with distributed responsibilities.

Table 4: Project Team Structure and Responsibilities

No.	**Team Member**	**Role**	**Primary Responsibilities**
1	Member 1	Product Owner / Backend Developer	Requirement definition, backlog management, backend processing, database integration
2	Member 2	SCRUM Master / Integration Coordinator	Sprint coordination, task tracking, Git management, subsystem integration
3	Member 3	Embedded Developer A (Sensors)	Sensor interfacing, ADC/I2C communication, firmware development
4	Member 4	Embedded Developer B (BLE Communication)	BLE advertising, packet transmission, gateway communication
5	Member 5	Frontend Dashboard Developer	Dashboard visualization, frontend implementation, real-time monitoring UI
6	Member 6	QA, Safety and Documentation Engineer	Testing, validation, safety verification, LaTeX documentation

The structure supports parallel subsystem development and iterative integration.

7.2 Management Reporting and Communication

The project follows SCRUM-based communication practices.

- Weekly sprint meetings
- Git-based Software Configuration Management
- Shared task tracking and milestone reviews
- Peer code reviews
- Continuous documentation updates

Technical blockers and risks are discussed during sprint reviews to ensure early mitigation and stable project progress.

8 Tracking and Control Mechanisms

8.1 Project Monitoring Strategy

Project progress is continuously monitored throughout the semester using SCRUM sprint planning, milestone tracking, sprint reviews, and integration validation procedures.

The distributed nature of the architecture requires careful coordination between embedded firmware development, backend integration, dashboard implementation, and testing activities.

8.2 Quality Assurance and Quality Control

Quality Assurance (QA) procedures are applied throughout both hardware and software development activities.

Hardware Quality Control includes:

- Sensor calibration verification
- Communication stability testing
- Gateway functionality validation
- Power and connectivity checks

Software Quality Assurance includes:

- Peer code reviews
- Incremental integration testing
- Backend validation procedures
- Dashboard verification
- Error handling validation

8.3 Software Configuration Management

The project utilizes Git as the primary Software Configuration Management (SCM) platform.

The repository structure separates firmware, backend, frontend, documentation, and testing components into independent modules.

The Git workflow consists of:

- `main` branch for stable releases
- `develop` branch for subsystem integration
- `feature/*` branches for isolated development tasks

Merge operations into stable branches require successful subsystem testing and peer review approval.

8.4 Progress Tracking

Sprint progress is monitored using:

- Product Backlog management
- Sprint Backlog tracking
- Weekly SCRUM meetings
- Sprint review demonstrations
- Sprint retrospectives
- Milestone validation checkpoints

Technical blockers and communication issues are documented during sprint meetings to ensure early risk identification and mitigation.

8.5 Testing and Validation Strategy

The testing process is divided into multiple validation phases:

8.5.1 Unit Testing

Individual firmware components, sensor drivers, and backend functions are validated independently before subsystem integration.

8.5.2 Integration Testing

Integration testing validates communication between:

- Sensor nodes and BLE gateway
- Gateway and backend server
- Backend server and dashboard interface

8.5.3 System Validation

Complete end-to-end system validation is performed using realistic monitoring scenarios and fault injection procedures.

Examples include:

- Disconnecting sensor nodes
- Simulating communication interruptions
- Validating timeout detection behavior
- Verifying dashboard alert generation

8.6 Documentation and Reporting

Project documentation is continuously maintained throughout the semester.

Documentation activities include:

- Sprint meeting minutes
- Architecture refinement
- Risk management updates
- Testing documentation
- Safety validation records
- Final report preparation

All official project documentation is maintained using LaTeX to ensure consistent formatting and professional presentation quality.

9 Literature Review

9.1 Indoor Environmental Monitoring

Indoor environmental quality has become an important research topic due to its direct impact on human health, comfort, and productivity. Research indicates that individuals spend a significant portion of their daily lives indoors, increasing the importance of continuous monitoring systems capable of detecting environmental changes and hazardous conditions.

IoT-based monitoring systems provide scalable and cost-effective approaches for collecting real-time environmental measurements using distributed wireless sensor networks.

9.2 IoT-Based Monitoring Architectures

Several recent studies propose distributed monitoring architectures utilizing low-cost embedded devices and wireless communication technologies.

John et al. developed an IoT-based monitoring framework for asthma-related environmental tracking capable of collecting environmental measurements within residential environments.

Rosa-Bilbao et al. proposed an intelligent indoor air quality management architecture utilizing distributed wireless nodes, backend processing, and event detection mechanisms for educational environments.

Simamora et al. explored mobile environmental monitoring approaches using autonomous IoT-based robotic systems capable of performing indoor environmental measurements dynamically.

9.3 Wireless Communication Technologies

The literature identifies several wireless communication technologies commonly used within IoT monitoring systems, including:

- Wi-Fi
- ZigBee
- LoRa
- Bluetooth Low Energy (BLE)

While Wi-Fi provides higher throughput and cloud connectivity, BLE offers lower power consumption and simpler deployment for localized wireless sensor networks.

The proposed architecture therefore adopts BLE communication using Nordic nRF52840 hardware in a star-topology configuration suitable for low-power indoor monitoring applications.

9.4 Sensor Technologies

Low-cost environmental sensing devices are widely used within IoT monitoring systems due to their accessibility and ease of integration.

The project utilizes an advanced multi-parameter suite:

- Sensirion SCD41 (CO_2, Temp, Humidity)
- Sensirion SPS30 (Particulate Matter)
- Adafruit CCS811 (TVOC, eCO_2)
- Microchip MCP9808 (High-precision temperature)
- KY-018 (Light intensity)

The literature demonstrates that low-cost sensors may exhibit calibration limitations and measurement inaccuracies. Consequently, software-level filtering and calibration procedures are important components of reliable environmental monitoring systems.

9.5 Safety and Reliability Considerations

Reliability remains a major concern within distributed wireless monitoring systems. Several studies discuss the importance of detecting communication failures, disconnected nodes, and sensor malfunctions.

The literature strongly supports the use of timeout monitoring, heartbeat detection, and backend-side fault analysis to prevent silent system failures.

These findings directly influence the fault detection strategies implemented within the proposed architecture.

9.6 AI-Assisted Research and Planning Tools

AI-assisted research tools including NotebookLM, Perplexity AI, and ChatGPT were used during literature exploration, architecture analysis, document refinement, and project planning activities.

All generated outputs were critically reviewed, refined, and adapted to ensure technical consistency with the project requirements and hardware architecture.

9.7 Documentation of LLM Interaction

Representative prompts used during literature analysis, architecture planning, project scheduling, risk management, and LaTeX document preparation are documented below.

Prompt 1 – Architecture Planning TODO

Paste your prompt here.

Prompt 2 – Literature Analysis

Create a Literature section for a Software Project Plan about a BLE-based indoor monitoring system using Nordic nRF52840 boards.
Use the uploaded research papers and project sources to create:

1. Preliminary literature overview
2. Summary of related indoor monitoring systems
3. Discussion of BLE and IoT communication approaches
4. Summary of sensor technologies
5. Safety and reliability considerations from the literature
6. Research limitations and future directions
7. Documentation of AI-assisted research and planning tools used during project development

Maintain academic writing style suitable for a Software Project Plan.

Prompt 3 – Project Planning

Paste your prompt here.

Prompt 4 – Risk Management

Create a project risk analysis for my BLE-based indoor monitoring system using nRF52840 boards.
Include: - technical risks - hardware risks - software risks - communication risks - safety and reliability concerns - project management risks
For each risk provide: 1. Risk description 2. Possible impact 3. Mitigation strategy 4. Probability level 5. Severity level
Present results in structured table format suitable for a university Software Project Plan.

Prompt 5 – Project Estimates

Create a Project Estimates section for a Software Project Plan about a BLE-based indoor monitoring system using Nordic nRF52840 boards.
The project includes: - distributed BLE sensor nodes - gateway node - backend server - dashboard visualization - temperature and light sensing
The development team consists of 6 members working in a SCRUM-based process during a semester-long project.
Include: 1. Historical assumptions used for estimation 2. Effort estimation approach 3. Estimated workload distribution 4. Time estimation 5. Project resources 6. Hardware and software resources 7. Reconciled project estimate
Maintain a professional software-engineering perspective and realistic implementation scope.
Use academic writing style suitable for a Software Project Plan.

Prompt 6 – Safety and Security Plan

Create a Safety and Security Plan section for a Software Project Plan about a BLE-based indoor monitoring system using Nordic nRF52840 boards.
The system includes: - distributed BLE sensor nodes - gateway node - backend server - dashboard visualization - temperature and light sensing
Include: 1. Safety objectives 2. Reliability considerations 3. Fault detection mechanisms 4. Communication reliability 5. Sensor reliability concerns 6. Security considerations 7. Data integrity and monitoring 8. Failure handling strategies
The project should emphasize: - reliability - monitoring - fault detection - communication integrity - risk management - system availability
Maintain a professional systems-engineering perspective while keeping the implementation complexity realistic for the proposed architecture.
Use academic writing style suitable for a Software Project Plan.

Prompt 7 – Project Schedule + Team Organization

Create a project schedule and SCRUM-based team organization for a university project developing a BLE-based indoor monitoring system using nRF52840 boards.
The project team consists of 6 students.
Include: 1. Suggested SCRUM roles 2. Team task distribution for 6 members 3. Functional decomposition of the project 4. Development milestones 5. Weekly implementation schedule for a semester-long project 6. Suggested Git repository organization 7. Testing and integration phases 8. Documentation and presentation preparation phases
The system includes: - BLE sensor nodes - gateway node - backend server - dashboard visualization - temperature and light sensing
Use academic project-planning style suitable for a Software Project Plan.

Prompt 8 – Tracking and Control Mechanisms

Create a Tracking and Control Mechanisms section for a Software Project Plan about a BLE-based indoor monitoring system using Nordic nRF52840 boards.
The project uses: - SCRUM-based development - distributed BLE sensor nodes - gateway node - backend server - dashboard visualization
Include: 1. Quality assurance and quality control 2. Progress tracking mechanisms 3. Sprint reviews and retrospectives 4. Software configuration management 5. Git workflow and version control 6. Change management procedures 7. Testing and validation tracking 8. Documentation and reporting practices
Maintain a professional software-engineering and systems-engineering perspective suitable for an academic Software Project Plan.

Prompt 9 –

Paste your prompt here.

Prompt 10 –

Paste your prompt here.

Prompt –

Paste your prompt here.

10 Appendix

10.1 Hardware Kit Inventory

The project hardware kit contains:

- 1x Nordic nRF52840 Development Kit (Sensor Node)
- 1x Sensirion SCD41 CO_2, Temperature, and Humidity Sensor
- 1x Sensirion SPS30 Particulate Matter Sensor
- 1x Adafruit CCS811 TVOC and eCO_2 Sensor
- 1x Microchip MCP9808 Temperature Sensor
- 1x KY-018 Light Intensity Sensor

10.2 Repository Structure

The Git repository is organized into the following components:

- `/firmware` – Sensor node Zephyr firmware
- `/gateway` – Python BLE-to-HTTP Gateway
- `/backend` – Spring Boot processing services
- `/analytics` – FastAPI ML anomaly detection service
- `/dashboard` – React frontend visualization
- `/docs` – LaTeX documentation and reports
- `/docker` – Docker Compose orchestration files

10.3 Sprint Documentation

Sprint documentation includes:

- Sprint planning notes
- Task allocation records
- Weekly progress reports
- Risk tracking updates
- Testing results
- Validation procedures

11 References

References

[1] R. John, R. R. Kureshi, D. Thakker, and B. K. Mishra, "Internet of Things (IoT) and Indoor Air Quality (IAQ) Monitoring in the Health Domain," in *Proceedings of the International Conference on Internet of Things*, ACM, 2021.

[2] Nordic Semiconductor, "nRF52840 Product Specification," 2023.

[3] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation Forest," *Proc. 8th IEEE Int. Conf. Data Mining*, pp. 413–422, 2008.

[4] World Health Organisation, "WHO Guidelines for Indoor Air Quality: Selected Pollutants," WHO Press, 2010.

[5] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *JMLR*, vol. 12, pp. 2825–2830, 2011.

[6] Sensirion AG, "SCD41 CO2 Sensor Datasheet," 2021.

[7] Sensirion AG, "SPS30 Particulate Matter Sensor Datasheet," 2020.

[8] S. Newman, "Building Microservices (2nd ed.)," O'Reilly Media, 2021.